

Тестирование приложения "Кекс"

В уроке «Навигация между экранами с помощью Compose» вы узнали, как добавить навигацию в приложение Compose с помощью компонента Jetpack Navigation Compose.

В приложении **Cupcake** есть несколько экранов, по которым можно перемещаться, и множество действий, которые может выполнять пользователь. Это приложение предоставляет вам отличную возможность отточить свои навыки автоматизированного тестирования! В этом уроке вы напишете несколько UI-тестов для приложения **Cupcake** и узнаете, как добиться максимального покрытия тестами.

Необходимые условия

- Знакомство с языком Kotlin, включая типы функций, лямбды и функции области видимости
- Завершение коделаба «Перемещение между экранами с помощью Compose».

Что вы узнаете

- Тестировать компонент Jetpack Navigation с помощью Compose.
- Создавать согласованное состояние пользовательского интерфейса для каждого теста пользовательского интерфейса.
- Создавать вспомогательные функции для тестов.

Что вы будете создавать

- UI-тесты для приложения Cupcake

Что вам понадобится

- Последняя версия Android Studio
- Подключение к интернету для загрузки стартового кода

2. Загрузите стартовый код

URL стартового кода:

<https://github.com/google-developer-training/basic-android-kotlin-compose-training-cupcake>

Название ветки со стартовым кодом: navigation

В Android Studio откройте папку basic-android-kotlin-compose-training-cupcake. Откройте код приложения Cupcake в Android Studio.

3. Настройка Cupcake для UI-тестов

Добавьте зависимости androidTest Инструмент сборки Gradle позволяет добавлять зависимости для определенных модулей. Эта функциональность позволяет избежать ненужной компиляции

зависимостей. Вы уже знакомы с конфигурацией внедрения при включении зависимостей в проект. Вы использовали это ключевое слово для импорта зависимостей в файле `build.gradle.kts` модуля `app`. Использование ключевого слова `implementation` делает эту зависимость доступной для всех наборов исходных текстов в этом модуле; на данном этапе курса вы получили опыт работы с наборами исходных текстов `main`, `test` и `androidTest`.

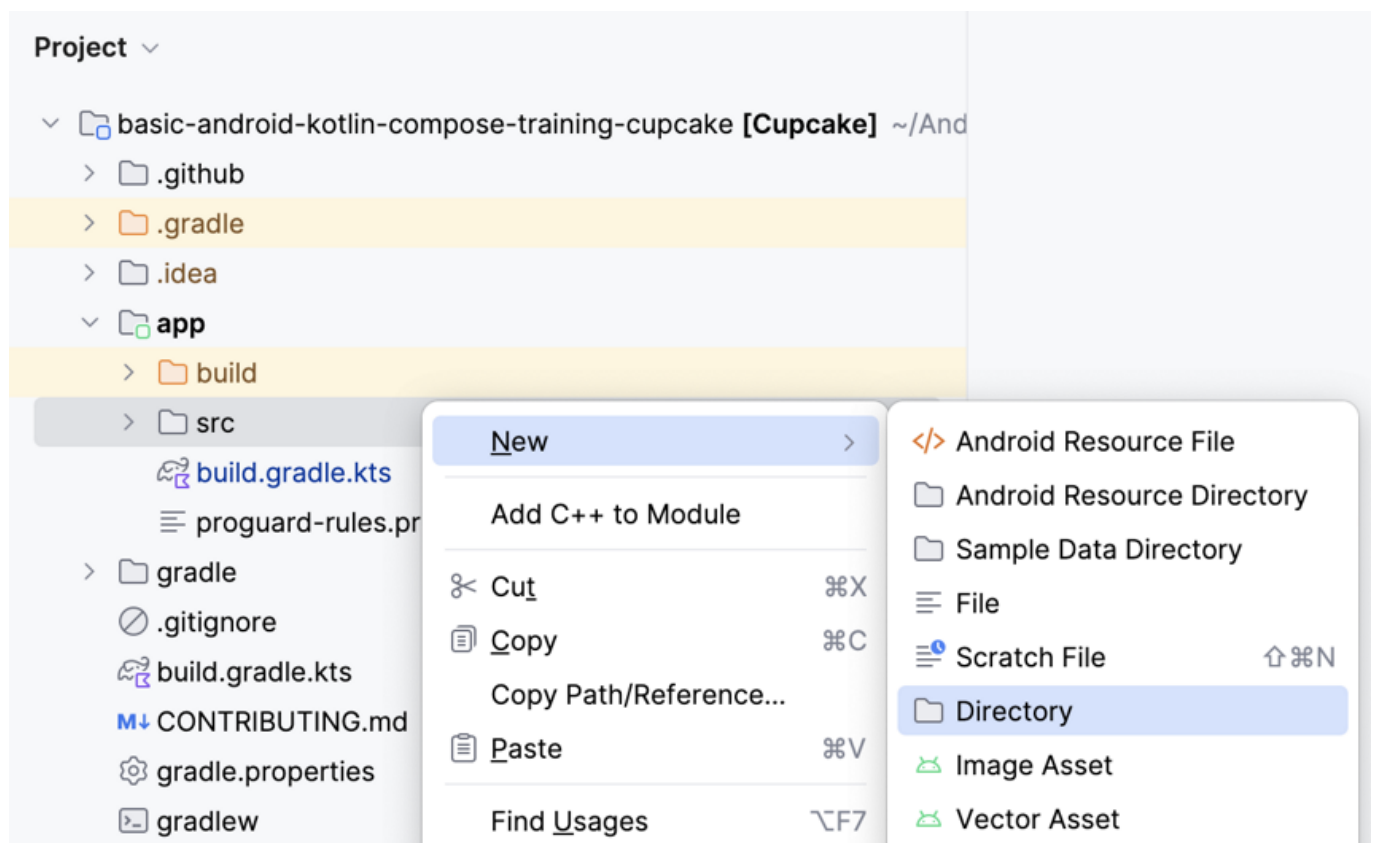
UI-тесты содержатся в собственном наборе исходных текстов под названием `androidTest`. Зависимости, которые нужны только для этого модуля, не нужно компилировать для других модулей, таких как главный модуль, в котором содержится код приложения. При добавлении зависимости, которая используется только UI-тестами, используйте ключевое слово `androidTestImplementation`, чтобы объявить зависимость в файле `build.gradle.kts` модуля приложения. Это гарантирует, что зависимости UI-тестов будут компилироваться только при запуске UI-тестов.

Выполните следующие шаги, чтобы добавить зависимости, необходимые для написания UI-тестов:

Откройте файл `build.gradle.kts` (Module :app). Добавьте следующие зависимости в раздел зависимостей файла:

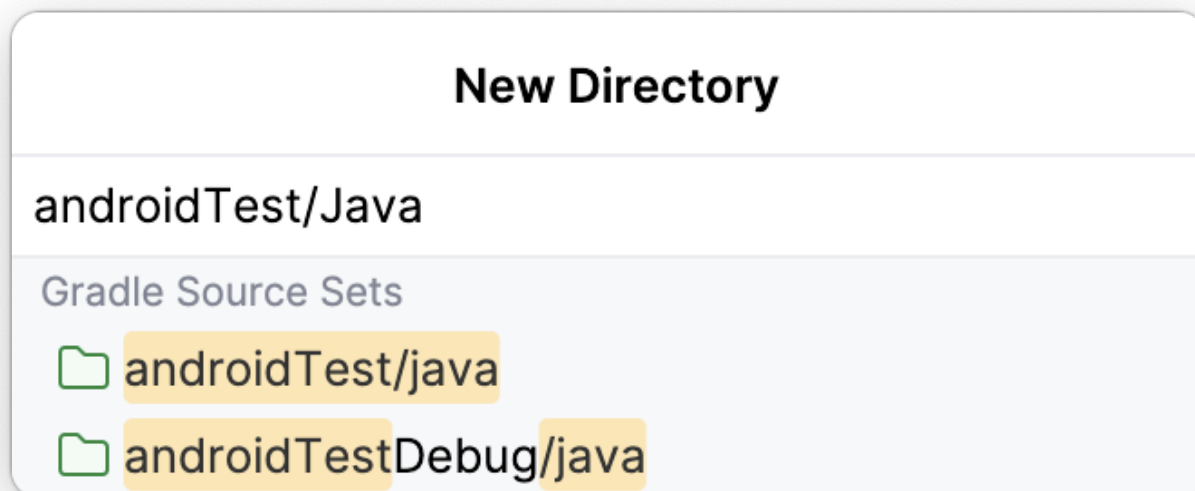
```
androidTestImplementation(platform("androidx.compose:compose-bom:2023.05.01"))
androidTestImplementation("androidx.compose.ui:ui-test-junit4")
androidTestImplementation("androidx.navigation:navigation-testing:2.6.0")
androidTestImplementation("androidx.test.espresso:espresso-intents:3.5.1")
androidTestImplementation("androidx.test.ext:junit:1.1.5")
```

Создайте директорию для тестирования пользовательского интерфейса Щелкните правой кнопкой мыши каталог `src` в представлении проекта и выберите `New > Directory`.

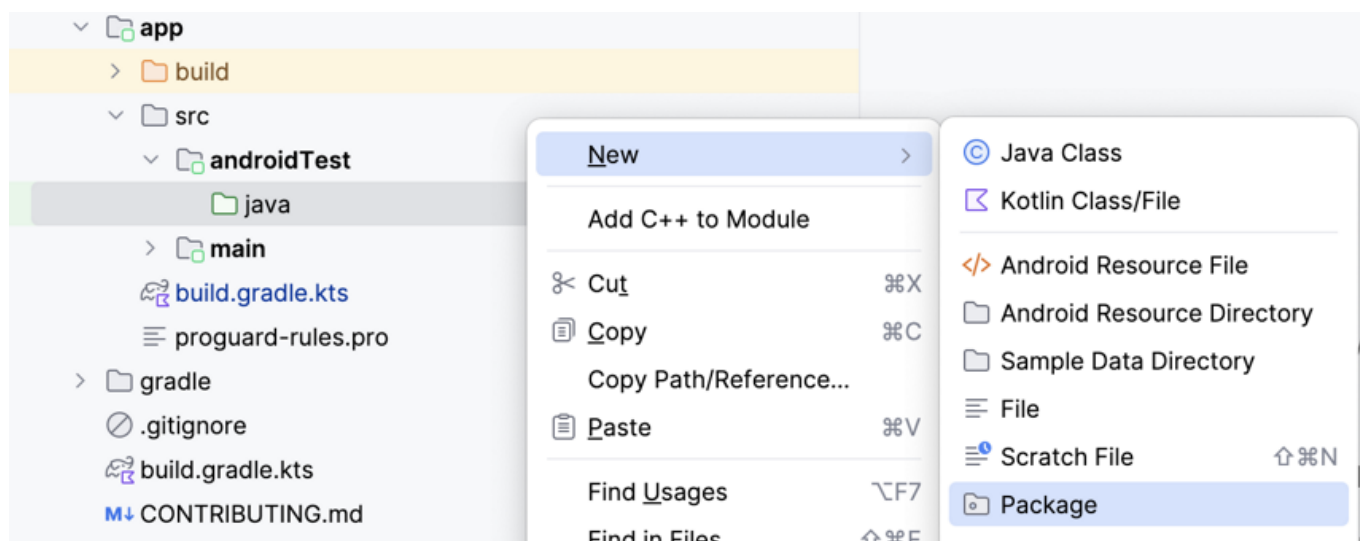


Выберите опцию androidTest/java.

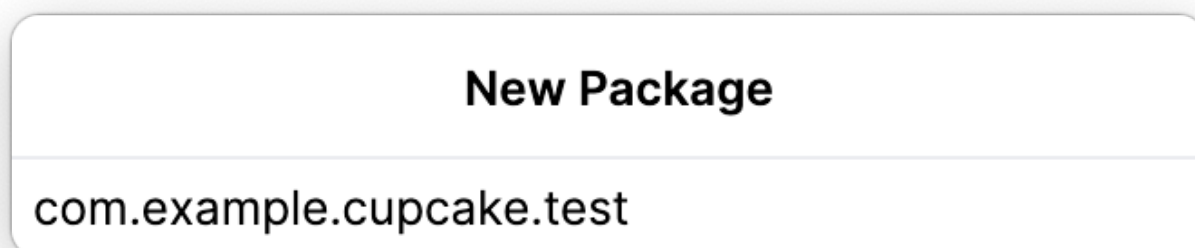
Примечание: Чтобы найти эту опцию, вам, возможно, придется прокрутить страницу вниз.



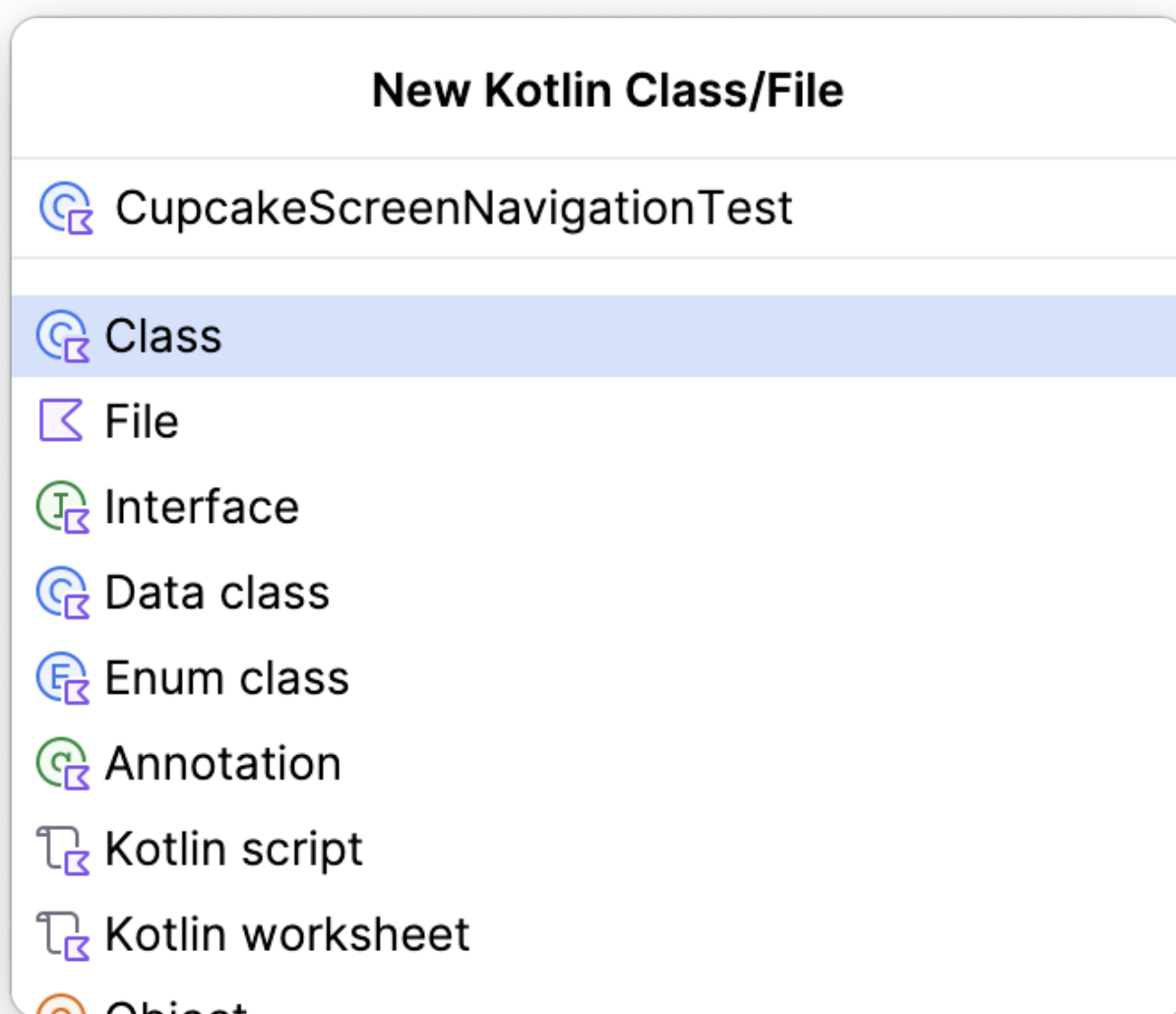
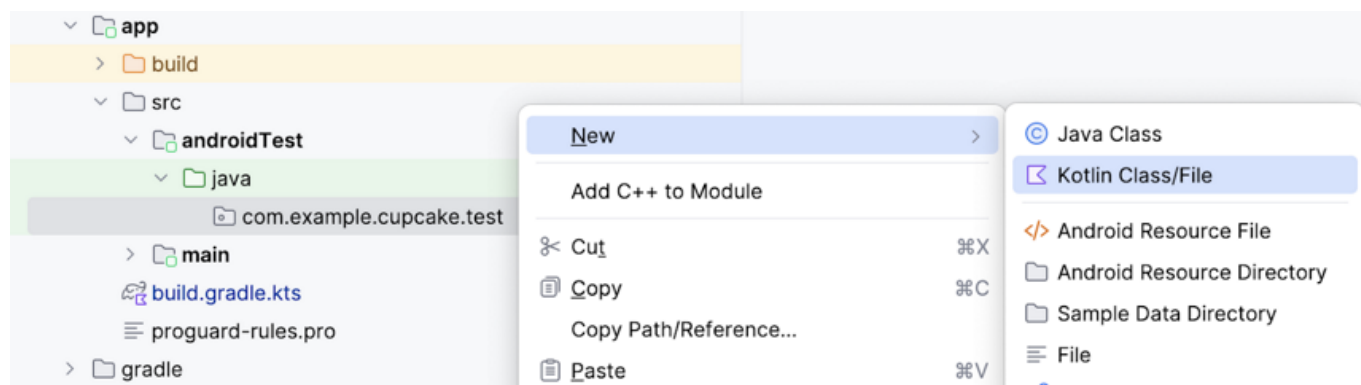
Создайте тестовый пакет Щелкните правой кнопкой мыши каталог androidTest/java в окне проекта и выберите New > Package.



Назовите пакет com.example.cupcake.test.



Создайте класс для тестирования навигации В директории test создайте новый Kotlin-класс CupcakeScreenNavigationTest.



4. Настройка узла навигации

В предыдущем коделабе вы узнали, что для тестирования пользовательского интерфейса в Compose требуется правило тестирования Compose. То же самое относится и к тестированию Jetpack Navigation.

Однако тестирование навигации требует дополнительных настроек с помощью правила тестирования Compose.

При тестировании навигации в Compose у вас не будет доступа к тому же контроллеру NavController, что и в коде приложения. Однако вы можете использовать TestNavController и настроить правило тестирования с помощью этого контроллера. В этом разделе вы узнаете, как настроить и повторно использовать тестовое правило для навигационных тестов.

В файле CupcakeScreenNavigationTest.kt создайте правило тестирования, используя createAndroidComposeRule и передав ComponentActivity в качестве параметра type.

```
import androidx.activity.ComponentActivity
import androidx.compose.ui.test.junit4.createAndroidComposeRule
import org.junit.Rule

@get:Rule
val composeTestRule = createAndroidComposeRule<ComponentActivity>()
```

Чтобы убедиться в том, что ваше приложение переходит в нужное место, необходимо сослаться на экземпляр TestNavController для проверки навигационного маршрута хоста навигации, когда приложение выполняет действия по переходу.

Инстанцируйте экземпляр TestNavController как переменную lateinit. В Kotlin ключевое слово lateinit используется для объявления свойства, которое может быть инициализировано после объявления объекта.

```
import androidx.navigation.testing.TestNavController

private lateinit var navController: TestNavController
```

Далее укажите компонент, который вы хотите использовать для UI-тестов.

Создайте метод setupCupcakeNavController(). В методе setupCupcakeNavController() вызовите метод setContent() для созданного вами тестового правила Compose. Внутри лямбды, переданной методу setContent(), вызовите композит CupcakeApp().

```
import com.example.cupcake.CupcakeApp

fun setupCupcakeNavController() {
    composeTestRule.setContent {
        CupcakeApp()
    }
}
```

Теперь вам нужно создать объект TestNavController в тестовом классе. Позже вы будете использовать этот объект для определения состояния навигации, поскольку приложение использует

контроллер для навигации по различным экранам в приложении Cupcake.

Настройте хост навигации с помощью лямбды, которую вы создали ранее. Инициализируйте созданную вами переменную `navController`, зарегистрируйте навигатор и передайте этот `TestNavHostController` в составной `CupcakeApp`.

```
import androidx.compose.ui.platform.LocalContext

fun setupCupcakeNavHost() {
    composeTestRule.setContent {
        navController = TestNavHostController(LocalContext.current).apply {
            navigatorProvider.addNavigator(ComposeNavigator())
        }
        CupcakeApp(navController = navController)
    }
}
```

Каждый тест в классе `CupcakeScreenNavigationTest` включает в себя проверку какого-либо аспекта навигации. Поэтому каждый тест зависит от созданного вами объекта `TestNavHostController`. Вместо того чтобы вручную вызывать функцию `setupCupcakeNavHost()` для каждого теста, чтобы настроить контроллер навигации, вы можете сделать это автоматически с помощью аннотации `@Before`, предоставляемой библиотекой `junit`. Когда метод аннотирован с `@Before`, он запускается перед каждым методом, аннотированным с `@Test`.

Добавьте аннотацию `@Before` в метод `setupCupcakeNavHost()`.

```
import org.junit.Before

@Before
fun setupCupcakeNavHost() {
    composeTestRule.setContent {
        navController = TestNavHostController(LocalContext.current).apply {
            navigatorProvider.addNavigator(ComposeNavigator())
        }
        CupcakeApp(navController = navController)
    }
}
```

5. Напишите навигационные тесты

Проверка начального пункта назначения. Вспомните, что при создании приложения Cupcake вы создали перечислительный класс `CupcakeScreen`, который содержал константы, определяющие навигацию по приложению.

`CupcakeScreen.kt`

```
/**
 * enum values that represent the screens in the app
 */
enum class CupcakeScreen(@StringRes val title: Int) {
    Start(title = R.string.app_name),
    Flavor(title = R.string.choose_flavor),
    Pickup(title = R.string.choose_pickup_date),
    Summary(title = R.string.order_summary)
}
```

У всех приложений с пользовательским интерфейсом есть домашний экран. Для Cupcake таким экраном является стартовый экран заказа. Контроллер навигации в составном CupcakeApp использует элемент Start перечисления CupcakeScreen, чтобы определить, когда переходить к этому экрану. При запуске приложения, если маршрут назначения еще не существует, маршрут назначения узла навигации устанавливается на CupcakeScreen.Start.name.

Сначала вам нужно написать тест, который проверит, что стартовый экран заказа является текущим маршрутом назначения при запуске приложения.

Создайте функцию cupcakeNavHost_verifyStartDestination() и аннотируйте ее с помощью @Test.

```
import org.junit.Test

@Test
fun cupcakeNavHost_verifyStartDestination() {
}
```

Теперь необходимо подтвердить, что начальным маршрутом назначения навигационного контроллера является начальный экран заказа.

Убедитесь, что ожидаемое имя маршрута (в данном случае CupcakeScreen.Start.name) равно маршруту назначения текущего элемента заднего стека навигационного контроллера.

```
import org.junit.Assert.assertEquals
...

@Test
fun cupcakeNavHost_verifyStartDestination() {
    assertEquals(CupcakeScreen.Start.name,
        navController.currentBackStackEntry?.destination?.route)
}
```

Примечание: Созданное вами правило AndroidComposeTestRule автоматически запускает приложение, отображая композитный CupcakeApp до выполнения любого метода @Test. Поэтому вам не нужно предпринимать никаких дополнительных шагов в тестовых методах для запуска приложения.

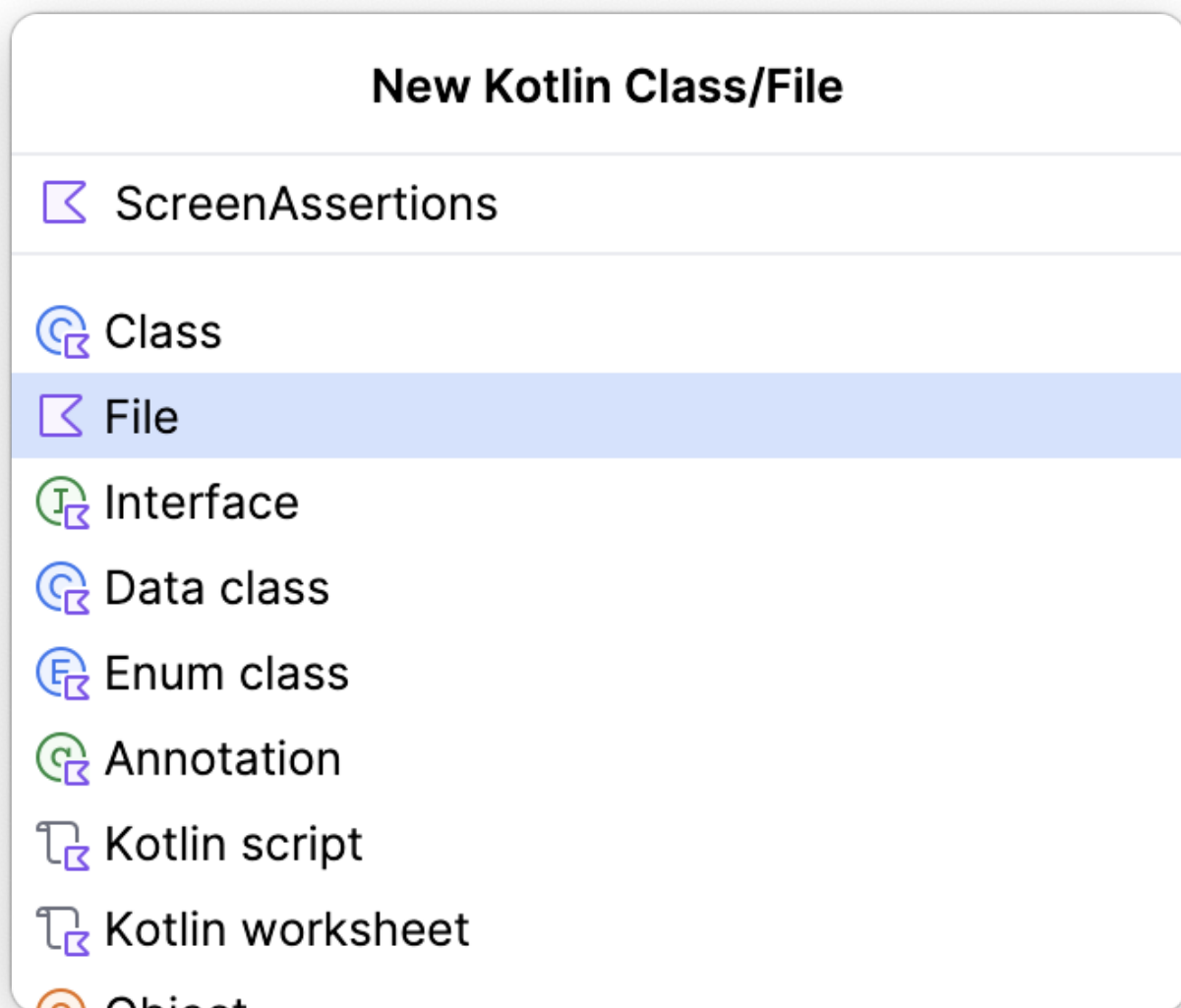
Создание вспомогательных методов Тесты пользовательского интерфейса часто требуют повторения шагов, чтобы привести пользовательский интерфейс в состояние, в котором можно протестировать определенную часть пользовательского интерфейса. Пользовательский пользовательский интерфейс также может требовать сложных утверждений, которые требуют много строк кода. Утверждение, которое вы написали в предыдущем разделе, требует много кода, и вы используете это же утверждение много раз, когда тестируете навигацию в приложении Cupcake. В таких ситуациях написание вспомогательных методов в тестах избавляет вас от написания дублирующего кода.

В каждом тесте навигации вы используете свойство `name` элементов перечисления `CupcakeScreen`, чтобы проверить правильность текущего маршрута назначения контроллера навигации. Вы пишете вспомогательную функцию, которую можно вызывать всякий раз, когда нужно сделать такое утверждение.

Для создания этой вспомогательной функции выполните следующие действия:

Создайте пустой Kotlin-файл в каталоге `test` с именем `ScreenAssertions`.

Примечание: Убедитесь, что вы создаете файл, а не класс. Этот файл будет использоваться для создания функции расширения, для которой потребуется пустой файл Kotlin.



Добавьте в класс `NavController` функцию расширения `assertCurrentRouteName()` и передайте в сигнатуре метода строку для ожидаемого имени маршрута.

```
fun NavController.assertCurrentRouteName(expectedRouteName: String) {  
  
}
```

В этой функции необходимо убедиться, что ожидаемое имя маршрута (`expectedRouteName`) равно маршруту назначения текущей записи заднего стека навигационного контроллера.

```
import org.junit.Assert.assertEquals  
...  
  
fun NavController.assertCurrentRouteName(expectedRouteName: String) {  
    assertEquals(expectedRouteName, currentBackStackEntry?.destination?.route)  
}
```

Откройте файл `CupcakeScreenNavigationTest` и измените функцию `cupcakeNavHost_verifyStartDestination()`, чтобы вместо длинного утверждения использовать вашу новую функцию расширения.

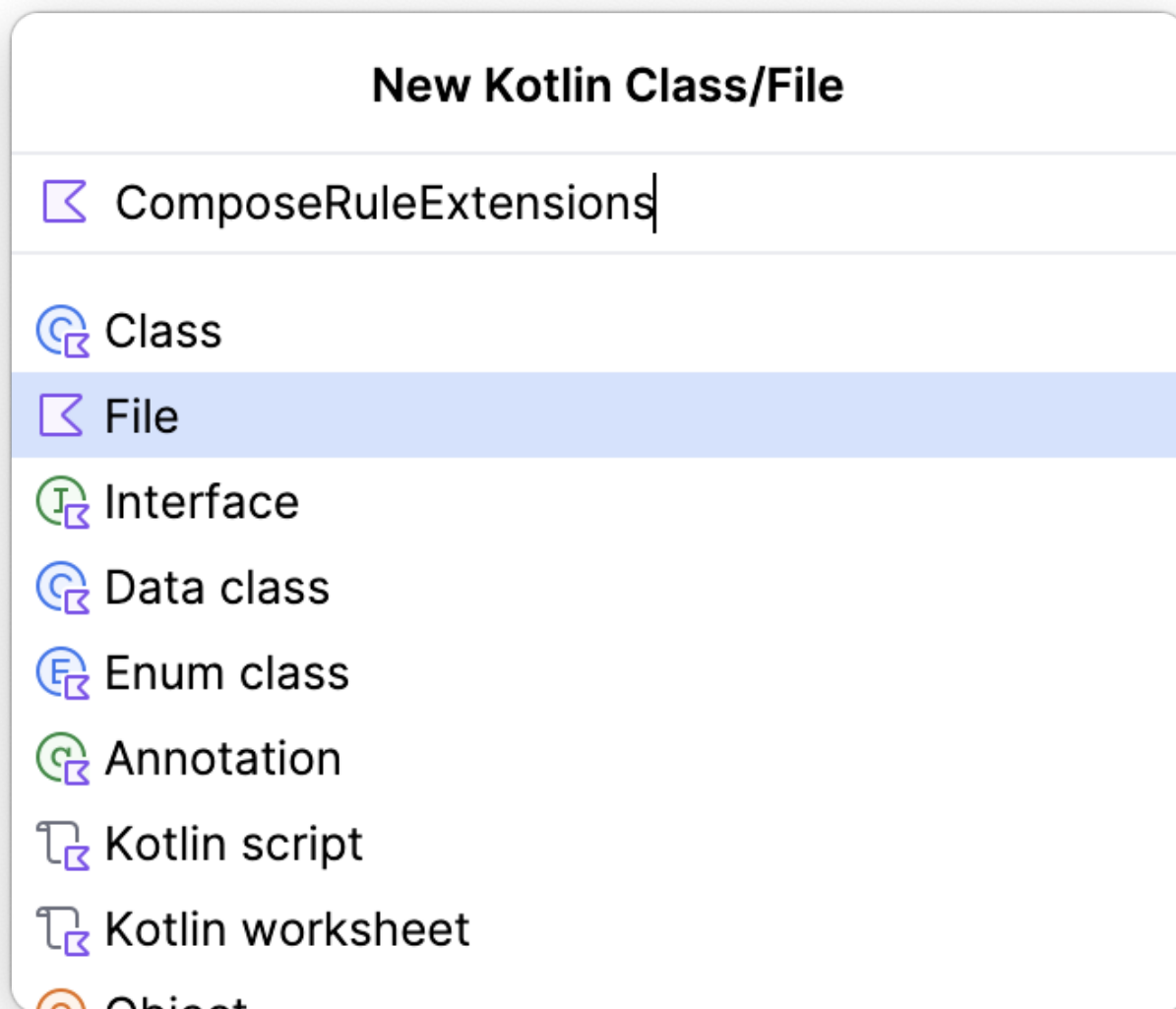
```
@Test  
fun cupcakeNavHost_verifyStartDestination() {  
    navController.assertCurrentRouteName(CupcakeScreen.Start.name)  
}
```

Ряд тестов также требует взаимодействия с компонентами пользовательского интерфейса. В этом коделабе такие компоненты часто находятся с помощью строки ресурсов. Получить доступ к `composable` по его ресурсной строке можно с помощью метода `Context.getString()`, о котором вы можете прочитать здесь. При написании UI-теста в `compose` реализация этого метода выглядит следующим образом:

```
composeTestRule.onNodeWithText(composeTestRule.activity.getString(R.string.my_string))
```

Это многословная инструкция, и ее можно упростить, добавив функцию расширения.

Создайте новый файл в пакете `com.example.cupcake.test` под названием `ComposeRuleExtensions.kt`. Убедитесь, что это обычный файл на языке Kotlin.



Добавьте в этот файл следующий код.

```
import androidx.activity.ComponentActivity
import androidx.annotation.StringRes
import androidx.compose.ui.test.SemanticsNodeInteraction
import androidx.compose.ui.test.junit4.AndroidComposeTestRule
import androidx.compose.ui.test.onNodeWithText
import androidx.test.ext.junit.rules.ActivityScenarioRule

fun <A : ComponentActivity> AndroidComposeTestRule<ActivityScenarioRule<A>,
A>.onNodeWithStringId(
    @StringRes id: Int
): SemanticsNodeInteraction = onNodeWithText(activity.getString(id))
```

Эта функция расширения позволяет сократить количество кода при поиске компонента пользовательского интерфейса по его строковому ресурсу. Вместо того чтобы писать это:

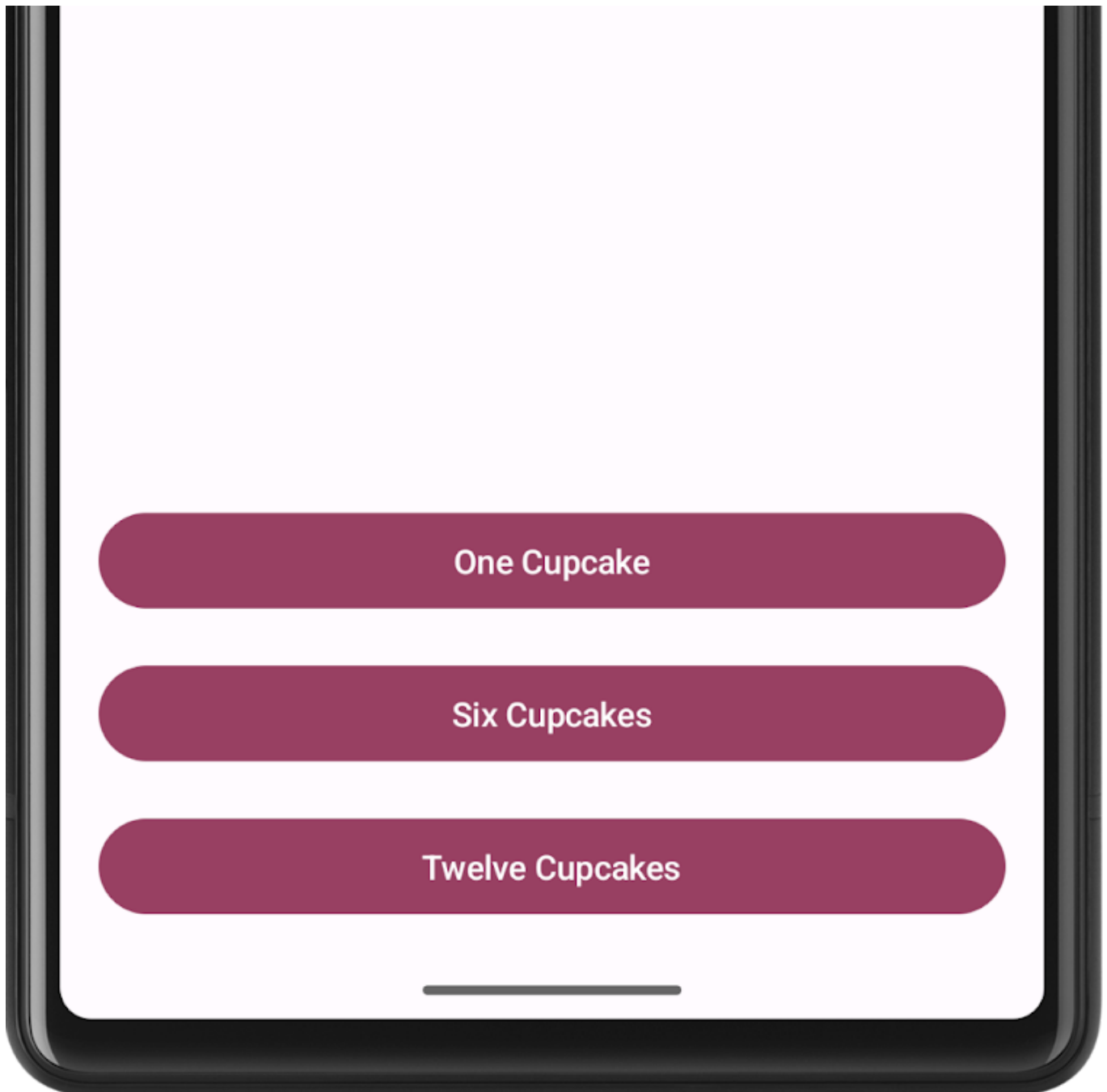
```
composeTestRule.onNodeWithText(composeTestRule.activity.getString(R.string.my_string))
```

Теперь вы можете воспользоваться следующей инструкцией:

```
composeTestRule.onNodeWithStringId(R.string.my_string)
```

Убедитесь, что на начальном экране нет кнопки «Вверх». В оригинальном дизайне приложения Cupcake на панели инструментов начального экрана отсутствует кнопка «Вверх».





На начальном экране нет кнопки, потому что с него некуда перейти вверх, так как это начальный экран. Выполните следующие шаги, чтобы создать функцию, которая подтверждает, что на начальном экране нет кнопки «Вверх»:

Создайте метод `cupcakeNavHost_verifyBackNavigationNotShownOnStartOrderScreen()` и аннотируйте его с помощью `@Test`.

```
@Test
fun cupcakeNavHost_verifyBackNavigationNotShownOnStartOrderScreen() {
}
```

В Cupcake для кнопки Up описание содержимого установлено в строку из ресурса `R.string.back_button`.

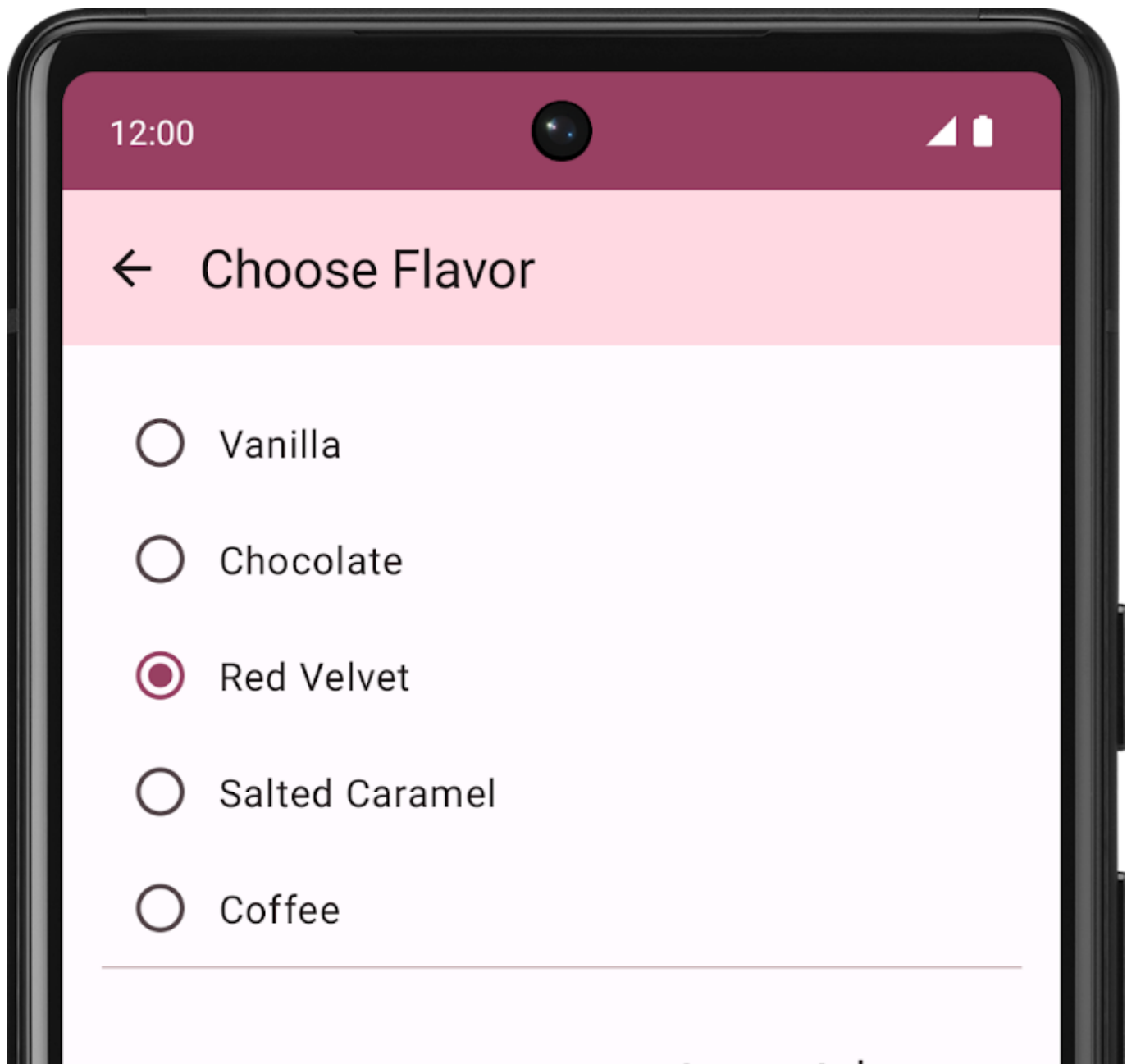
Создайте переменную в тестовой функции со значением ресурса `R.string.back_button`.

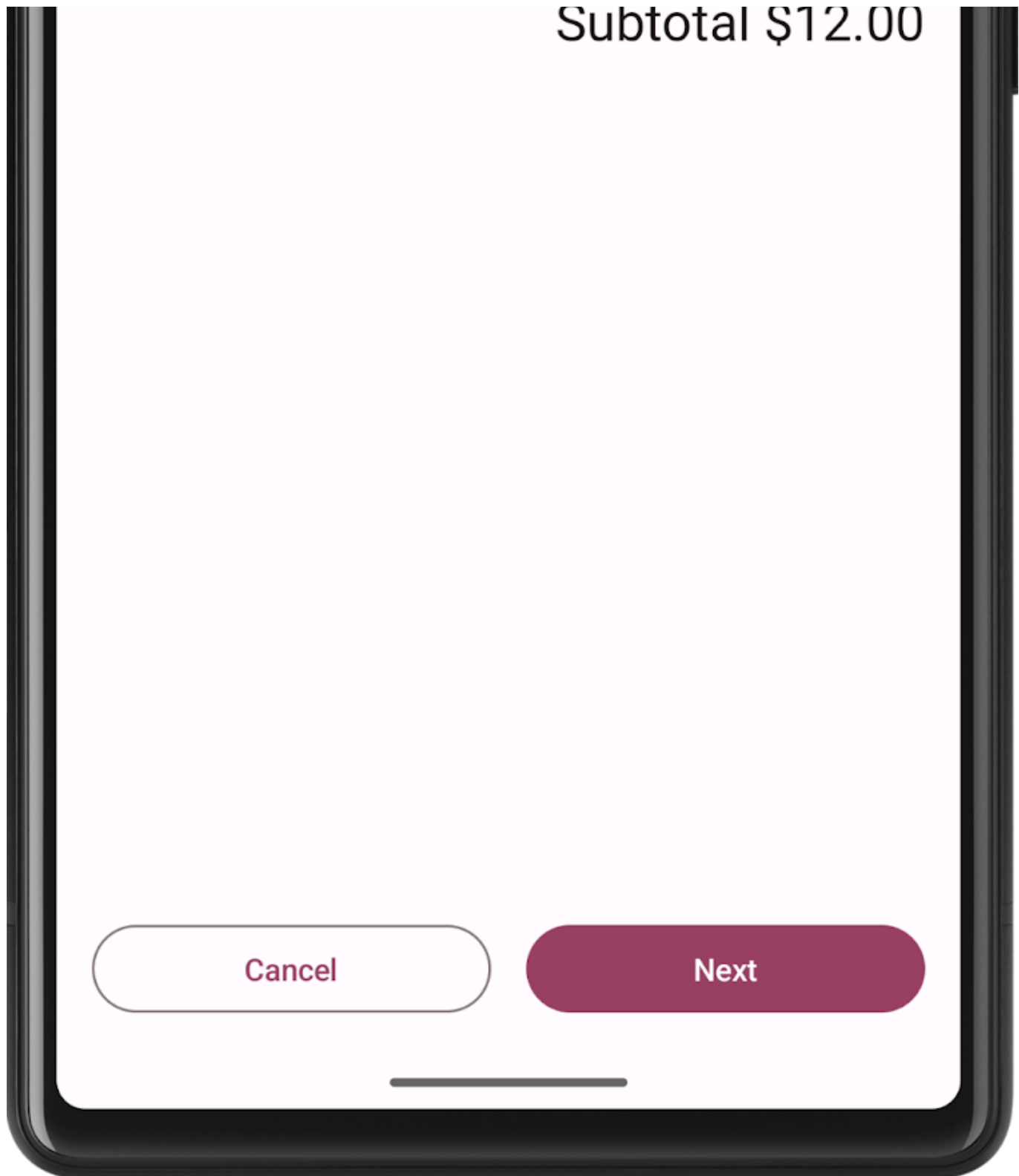
```
@Test
fun cupcakeNavHost_verifyBackNavigationNotShownOnStartOrderScreen() {
    val backText = composeTestRule.activity.getString(R.string.back_button)
}
```

Утверждение, что узел с таким описанием содержимого не существует на экране.

```
@Test
fun cupcakeNavHost_verifyBackNavigationNotShownOnStartOrderScreen() {
    val backText = composeTestRule.activity.getString(R.string.back_button)
    composeTestRule.onNodeWithContentDescription(backText).assertDoesNotExist()
}
```

Проверка навигации к экрану Flavor При нажатии на одну из кнопок на начальном экране запускается метод, который указывает навигационному контроллеру перейти к экрану Flavor.





В этом тесте вы напишете команду для нажатия кнопки, чтобы запустить эту навигацию, и проверите, что конечным маршрутом является экран Flavor.

Создайте функцию `cupcakeNavHost_clickOneCupcake_navigatesToSelectFlavorScreen()` и аннотируйте ее с помощью `@Test`.

```
@Test
fun cupcakeNavHost_clickOneCupcake_navigatesToSelectFlavorScreen(){
}
```

Примечание: Имена тестовых методов отличаются от имен функций, которые вы используете в коде приложения. Хорошим соглашением об именовании тестовых методов является следующее: `thingUnderTest_TriggerOfTest_ResultOfTest`. На примере функции, которую вы только что создали, ясно, что вы тестируете хост навигации по кексам, нажимая кнопку One Cupcake, и ожидаемым результатом является переход на экран Flavor.

Найдите кнопку One Cupcake по ее строковому идентификатору ресурса и выполните действие щелчка по ней.

```
import com.example.cupcake.R
...

@Test
fun cupcakeNavHost_clickOneCupcake_navigatesToSelectFlavorScreen() {
    composeTestRule.onNodeWithStringId(R.string.one_cupcake)
        .performClick()
}
```

Убедитесь, что текущее имя маршрута - это имя экрана Flavor.

```
@Test
fun cupcakeNavHost_clickOneCupcake_navigatesToSelectFlavorScreen() {
    composeTestRule.onNodeWithStringId(R.string.one_cupcake)
        .performClick()
    navController.assertCurrentRouteName(CupcakeScreen.Flavor.name)
}
```

Напишите больше вспомогательных методов Приложение Cupcake имеет в основном линейную навигацию. За исключением нажатия кнопки «Отмена», вы можете перемещаться по приложению только в одном направлении. Поэтому, тестируя экраны, расположенные глубже в приложении, вы можете столкнуться с необходимостью повторять код для перехода к тем областям, которые вы хотите протестировать. В такой ситуации стоит использовать больше вспомогательных методов, чтобы код можно было написать только один раз.

Теперь, когда вы протестировали навигацию к экрану Flavor, создайте метод, который переходит к экрану Flavor, чтобы не повторять этот код в будущих тестах.

Примечание: Помните, что это не тестовые методы. Они ничего не тестируют и должны выполняться только при явном вызове. Поэтому они не должны быть аннотированы с `@Test`.

Create a method called `navigateToFlavorScreen()`.

```
private fun navigateToFlavorScreen() {
}
```

Напишите команду для поиска кнопки One Cupcake и выполнения действия щелчка по ней, как вы делали в предыдущем разделе.

```
private fun navigateToFlavorScreen() {
    composeTestRule.onNodeWithStringId(R.string.one_cupcake)
        .performClick()
}
```

Помните, что кнопка «Далее» на экране «Аромат» не будет доступна для нажатия, пока не будет выбран аромат. Этот метод предназначен только для подготовки пользовательского интерфейса к навигации. После вызова этого метода пользовательский интерфейс должен находиться в состоянии, в котором кнопка «Далее» будет доступна для нажатия.

Найдите в пользовательском интерфейсе узел со строкой `R.string.chocolate` и выполните на нем действие щелчка, чтобы выбрать его.

```
private fun navigateToFlavorScreen() {
    composeTestRule.onNodeWithStringId(R.string.one_cupcake)
        .performClick()
    composeTestRule.onNodeWithStringId(R.string.chocolate)
        .performClick()
}
```

Попробуйте написать вспомогательные методы для перехода к экрану Pickup и экрану Summary. Попробуйте выполнить это упражнение самостоятельно, прежде чем обратиться к решению.

Примечание: Для перехода к экрану сводки сначала нужно выбрать дату на экране Pickup. Вам необходимо сгенерировать дату для выбора в пользовательском интерфейсе.

Для этого используйте следующий код:

```
private fun getFormattedDate(): String {
    val calendar = Calendar.getInstance()
    calendar.add(java.util.Calendar.DATE, 1)
    val formatter = SimpleDateFormat("E MMM d", Locale.getDefault())
    return formatter.format(calendar.time)
}
```

```
private fun navigateToPickupScreen() {
    navigateToFlavorScreen()
    composeTestRule.onNodeWithStringId(R.string.next)
        .performClick()
}

private fun navigateToSummaryScreen() {
```



```

        navigateToPickupScreen()
        composeTestRule.onNodeWithText(getFormattedDate())
            .performClick()
        composeTestRule.onNodeWithStringId(R.string.next)
            .performClick()
    }

```

При тестировании экранов, выходящих за пределы начального экрана, необходимо предусмотреть проверку функциональности кнопки «Вверх», чтобы убедиться, что она направляет навигацию к предыдущему экрану. Рассмотрите возможность создания вспомогательной функции для поиска и нажатия кнопки «Вверх».

```

private fun performNavigateUp() {
    val backText = composeTestRule.activity.getString(R.string.back_button)
    composeTestRule.onNodeWithContentDescription(backText).performClick()
}

```

Максимальное покрытие тестами Набор тестов приложения должен проверять как можно больше функциональности приложения. В идеальном мире набор тестов UI должен покрывать 100 % функциональности пользовательского интерфейса. На практике такого покрытия тестами добиться сложно, поскольку существует множество внешних по отношению к вашему приложению факторов, которые могут повлиять на пользовательский интерфейс, например, устройства с уникальным размером экрана, различные версии операционной системы Android, а также сторонние приложения, которые могут повлиять на другие приложения на телефоне.

Один из способов увеличить покрытие тестами - писать тесты вместе с функциями по мере их добавления. Таким образом, вы избежите слишком большого опережения в работе над новыми функциями и необходимости возвращаться назад, чтобы вспомнить все возможные сценарии. На данный момент Cupcake - довольно небольшое приложение, и вы уже протестировали значительную часть навигации приложения! Однако есть еще несколько состояний навигации, которые необходимо протестировать.

Попробуйте написать тесты для проверки следующих состояний навигации. Попробуйте реализовать их самостоятельно, прежде чем смотреть на решение.

Переход на начальный экран при нажатии кнопки «Вверх» с экрана «Вкус Переход к экрану «Старт» при нажатии кнопки «Отмена» на экране «Вкус Переход к экрану «Подборка Переход к экрану «Аромат» осуществляется нажатием кнопки «Вверх» на экране «Подборка». Переход к экрану «Начало» осуществляется нажатием кнопки «Отмена» на экране «Подборка». Переход к экрану Сводка Переход к начальному экрану осуществляется нажатием кнопки «Отмена» на экране «Сводка»

```

@Test
fun cupcakeNavHost_clickNextOnFlavorScreen_navigatesToPickupScreen() {
    navigateToFlavorScreen()
    composeTestRule.onNodeWithStringId(R.string.next)
        .performClick()
    navController.assertCurrentRouteName(CupcakeScreen.Pickup.name)
}

```

```
}

@Test
fun cupcakeNavHost_clickBackOnFlavorScreen_navigatesToStartOrderScreen() {
    navigateToFlavorScreen()
    performNavigateUp()
    navController.assertCurrentRouteName(CupcakeScreen.Start.name)
}

@Test
fun cupcakeNavHost_clickCancelOnFlavorScreen_navigatesToStartOrderScreen() {
    navigateToFlavorScreen()
    composeTestRule.onNodeWithStringId(R.string.cancel)
        .performClick()
    navController.assertCurrentRouteName(CupcakeScreen.Start.name)
}

@Test
fun cupcakeNavHost_clickNextOnPickupScreen_navigatesToSummaryScreen() {
    navigateToPickupScreen()
    composeTestRule.onNodeWithText(getFormattedDate())
        .performClick()
    composeTestRule.onNodeWithStringId(R.string.next)
        .performClick()
    navController.assertCurrentRouteName(CupcakeScreen.Summary.name)
}

@Test
fun cupcakeNavHost_clickBackOnPickupScreen_navigatesToFlavorScreen() {
    navigateToPickupScreen()
    performNavigateUp()
    navController.assertCurrentRouteName(CupcakeScreen.Flavor.name)
}

@Test
fun cupcakeNavHost_clickCancelOnPickupScreen_navigatesToStartOrderScreen() {
    navigateToPickupScreen()
    composeTestRule.onNodeWithStringId(R.string.cancel)
        .performClick()
    navController.assertCurrentRouteName(CupcakeScreen.Start.name)
}

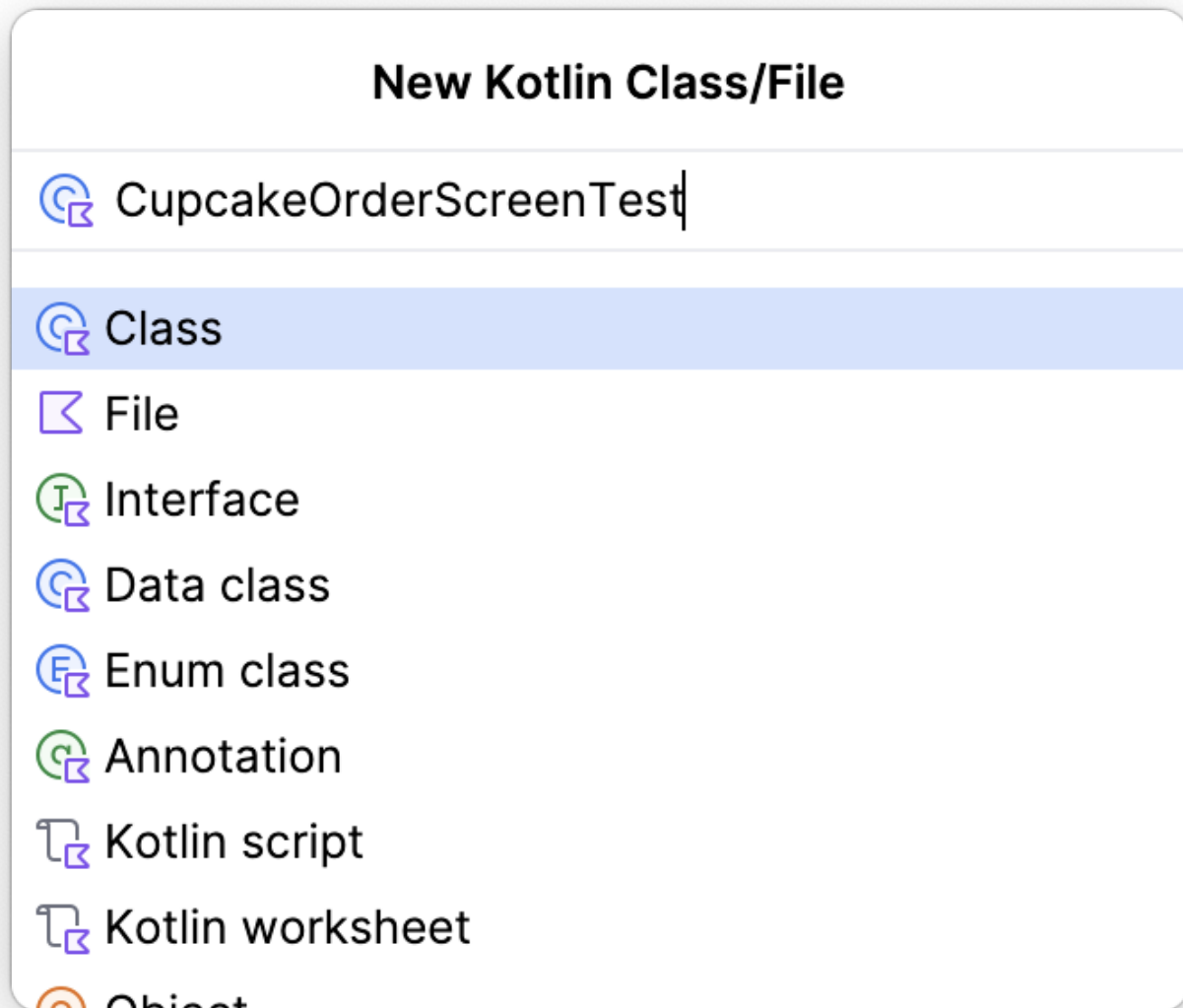
@Test
fun cupcakeNavHost_clickCancelOnSummaryScreen_navigatesToStartOrderScreen() {
    navigateToSummaryScreen()
    composeTestRule.onNodeWithStringId(R.string.cancel)
        .performClick()
    navController.assertCurrentRouteName(CupcakeScreen.Start.name)
}
```

6. Напишите тесты для экрана заказа

Навигация - это лишь один из аспектов функциональности приложения Cupcake. Пользователь также взаимодействует с каждым из экранов приложения. Вам нужно проверить, что отображается на этих экранах и что действия, выполняемые на них, приводят к правильным результатам. Экран `SelectOptionScreen` является важной частью приложения.

В этом разделе вы напишете тест, который проверит, правильно ли задано содержимое этого экрана.

Проверка содержимого экрана `Choose Flavor` Создайте новый класс в каталоге `app/src/androidTest` под названием `CupcakeOrderScreenTest`, где находятся и другие файлы тестов.



В этом классе создайте правило `AndroidComposeTestRule`.

```
@get:Rule
val composeTestRule = createAndroidComposeRule<ComponentActivity>()
```

Создайте функцию `selectOptionScreen_verifyContent()` и аннотируйте ее с помощью `@Test`.

```
@Test
fun selectOptionScreen_verifyContent() {

}
```

В этой функции вы задаете содержимое правила Compose для экрана SelectOptionScreen. Благодаря этому композитный экран SelectOptionScreen запускается напрямую, поэтому навигация не требуется. Однако этот экран требует двух параметров: списка вариантов вкуса и промежуточного итога.

Создайте список вариантов вкуса и промежуточный итог, которые будут переданы на экран.

```
@Test
fun selectOptionScreen_verifyContent() {
    // Given list of options
    val flavors = listOf("Vanilla", "Chocolate", "Hazelnut", "Cookie", "Mango")
    // And subtotal
    val subtotal = "$100"
}
```

Установите содержимое композита SelectOptionScreen, используя значения, которые вы только что создали. Обратите внимание, что этот подход похож на запуск композита из MainActivity. Разница лишь в том, что MainActivity вызывает композит CupcakeApp, а здесь вы вызываете композит SelectOptionScreen. Возможность изменить составной элемент, запускаемый с помощью функции setContent(), позволяет запускать определенные составные элементы вместо того, чтобы тест явно проходил через приложение, чтобы добраться до области, которую вы хотите протестировать. Такой подход помогает предотвратить сбой теста в тех областях кода, которые не имеют отношения к текущему тесту.

```
@Test
fun selectOptionScreen_verifyContent() {
    // Given list of options
    val flavors = listOf("Vanilla", "Chocolate", "Hazelnut", "Cookie", "Mango")
    // And subtotal
    val subtotal = "$100"

    // When SelectOptionScreen is loaded
    composeTestRule.setContent {
        SelectOptionScreen(subtotal = subtotal, options = flavors)
    }
}
```

На этом этапе теста приложение запускает составной элемент SelectOptionScreen, и вы можете взаимодействовать с ним с помощью тестовых инструкций.

Пройдитесь по списку вкусов и убедитесь, что каждый строковый элемент в списке отображается на экране. Используйте метод onNodeWithText(), чтобы найти текст на экране, и используйте метод

`assertIsDisplayed()`, чтобы убедиться, что текст отображается в приложении.

```
@Test
fun selectOptionScreen_verifyContent() {
    // Given list of options
    val flavors = listOf("Vanilla", "Chocolate", "Hazelnut", "Cookie", "Mango")
    // And subtotal
    val subtotal = "$100"

    // When SelectOptionScreen is loaded
    composeTestRule.setContent {
        SelectOptionScreen(subtotal = subtotal, options = flavors)
    }

    // Then all the options are displayed on the screen.
    flavors.forEach { flavor ->
        composeTestRule.onNodeWithText(flavor).assertIsDisplayed()
    }
}
```

Используя ту же методику проверки отображения текста, проверьте, что приложение выводит на экран правильную строку subtotal. Выполните поиск на экране идентификатора ресурса `R.string.subtotal_price` и правильного значения промежуточного итога, а затем убедитесь, что приложение отображает это значение.

```
import com.example.cupcake.R
...

@Test
fun selectOptionScreen_verifyContent() {
    // Given list of options
    val flavors = listOf("Vanilla", "Chocolate", "Hazelnut", "Cookie", "Mango")
    // And subtotal
    val subtotal = "$100"

    // When SelectOptionScreen is loaded
    composeTestRule.setContent {
        SelectOptionScreen(subtotal = subtotal, options = flavors)
    }

    // Then all the options are displayed on the screen.
    flavors.forEach { flavor ->
        composeTestRule.onNodeWithText(flavor).assertIsDisplayed()
    }

    // And then the subtotal is displayed correctly.
    composeTestRule.onNodeWithText(
        composeTestRule.activity.getString(
            R.string.subtotal_price,
            subtotal
        )
    ).assertIsDisplayed()
}
```

```

    )
    ).assertIsDisplayed()
}

```

Напомним, что кнопка «Далее» не включается, пока не выбран элемент. Этот тест проверяет только содержимое экрана, поэтому последнее, что нужно проверить, - это то, что кнопка «Далее» отключена.

Найдите кнопку «Далее», используя тот же подход к поиску узла по строковому идентификатору ресурса. Однако вместо того, чтобы проверять, отображает ли приложение узел, используйте метод `assertIsNotEnabled()`.

```

@Test
fun selectOptionScreen_verifyContent() {
    // Given list of options
    val flavors = listOf("Vanilla", "Chocolate", "Hazelnut", "Cookie", "Mango")
    // And subtotal
    val subtotal = "$100"

    // When SelectOptionScreen is loaded
    composeTestRule.setContent {
        SelectOptionScreen(subtotal = subtotal, options = flavors)
    }

    // Then all the options are displayed on the screen.
    flavors.forEach { flavor ->
        composeTestRule.onNodeWithText(flavor).assertIsDisplayed()
    }

    // And then the subtotal is displayed correctly.
    composeTestRule.onNodeWithText(
        composeTestRule.activity.getString(
            R.string.subtotal_price,
            subtotal
        )
    ).assertIsDisplayed()

    // And then the next button is disabled
    composeTestRule.onNodeWithStringId(R.string.next).assertIsNotEnabled()
}

```

Максимальное покрытие тестами Тест содержимого экрана Choose Flavor проверяет только один аспект одного экрана. Существует ряд дополнительных тестов, которые можно написать, чтобы увеличить покрытие кода. Попробуйте написать следующие тесты самостоятельно, прежде чем загружать код решения.

Проверка содержимого начального экрана. Проверьте содержимое экрана «Сводка». Проверьте, включается ли кнопка Next при выборе опции на экране Choose Flavor. При написании тестов не забывайте о вспомогательных функциях, которые могут сократить объем написанного кода!

Получение кода решения

Чтобы загрузить код готового урока, вы можете использовать эту команду git:

```
git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-cupcake.git
```

Резюме

Вы узнали, как тестировать компонент **Jetpack Navigation**. Вы также приобрели некоторые фундаментальные навыки написания UI-тестов, такие как написание многократно используемых вспомогательных методов, использование функции `setContent()` для написания кратких тестов, настройка тестов с помощью аннотации **@Before**, а также умение думать о максимальном покрытии тестами. Продолжая создавать приложения для Android, не забывайте писать тесты вместе с функциональным кодом!